

DOCUMENTATION FONCTIONNELLE

Systeme Multi-Agents de Gestion d'Emails

Version 2.0 - Avril 2026

Auteur

Bilimbilim Mombo

Contexte

Stage Technique

Ce document decrit le fonctionnement complet du systeme d'agents intelligents developpe pour la gestion automatisee des emails professionnels.

1. Présentation Générale du Système

1.1 Objectif du projet

Ce projet développe un système multi-agents capable de traiter automatiquement les emails professionnels reçus dans une boîte Gmail, en s'appuyant sur des modèles de langage locaux (LLM) et des architectures agentiques.

Le système est capable de :

- Lire les emails non lus depuis Gmail via l'API Google
- Analyser sémantiquement le contenu de chaque email
- Router le traitement vers l'agent spécialisé approprié
- Générer une réponse contextualisée et la renvoyer automatiquement
- Gérer les rendez-vous en lien avec Google Calendar
- Memoriser les échanges passés pour personnaliser les réponses futures

1.2 Technologies utilisées

Backend & Orchestration	Intégrations & Données
Python 3.11+	Google Gmail API
LangChain / LangGraph	Google Calendar API
Ollama (llama3)	ChromaDB (vectoriel)
FastAPI	Sentence-Transformers
Streamlit	Watchdog

1.3 Architecture globale

Pipeline d'exécution principal

PERCEPTION -> ANALYSE -> AGENTS SPECIALISES -> ACTION

Chaque email non lu transite séquentiellement par ces 4 étapes.
L'orchestration est réalisée par un graphe LangGraph compilé.

2. Architecture Détaillée

2.1 Structure des fichiers

Fichier	Rôle	Responsabilité
main_agent.py	Point d'entrée principal	Orchestre le pipeline complet
perception.py	Module PERCEPTION	Lecture et extraction des emails Gmail
analyse.py	Module ANALYSE	Classification LLM de l'intention
actions.py	Module ACTION	Envoi, trace et memoire
agent_email.py	Agent Email	Génération de réponses contextuelles
calendar_agent.py	Agent Calendrier	Gestion des rendez-vous
agent_rag.py	Agent RAG	Réponses documentaires
state.py	Modèles de données	AgentState, EmailModel, DecisionModel
gmail_auth.py	Authentification	OAuth2 Gmail & Calendar
app.py	API REST FastAPI	Interface web complète
ui_app.py	Dashboard Streamlit	Interface utilisateur graphique
metrics.py	Métriques qualité	Évaluation des réponses générées
attachments_analysis.py	Pièces jointes	Extraction et classification PDF
visualizer_server.py	Visualiseur WebSocket	Suivi en temps réel

2.2 Modèle de données (state.py)

L'état de l'agent circule dans un objet AgentState Pydantic partagé entre tous les nœuds du graphe.

EmailModel

- **sender_name** : Nom de l'expéditeur extrait du champ From
- **sender_email** : Adresse email de l'expéditeur
- **subject** : Objet de l'email
- **content** : Corps texte décodé (UTF-8)
- **conversation_history** : Historique des messages du thread Gmail

DecisionModel

- **intent** : calendar | email | document | general | unknown
- **priority** : low | normal | high | urgent

- **action** : reply | archive | review

AgentState

- **email** : Instance EmailModel
- **attachments** : Liste des chemins de pieces jointes telechargees
- **gmail_message_id** : Identifiant du message Gmail en cours
- **gmail_thread_id** : Identifiant du thread de conversation
- **decision** : Instance DecisionModel
- **result** : Message de resultat final

3. Pipeline de Traitement Detaille

3.1 Etape 1 - PERCEPTION (perception.py)

Le module de perception est le premier noeud du graphe LangGraph. Son role est de lire un email Gmail et de peupler l'AgentState avec les donnees structurees de l'email.

Fonctions cles

- **fetch_all_unread_ids()** : Recupere de facon paginee tous les IDs des emails non lus (filtre : is:inbox is:unread) depuis l'API Gmail.
- **read_email_gmail()** : Lit un message precis via son ID Gmail, extrait les headers (From, Subject, Date), decode le corps en base64 et charge l'historique du thread.
- **extract_body()** : Gere les emails simples et multipart. Cherche recursivement la part text/plain dans les structures imbriquees MIME.
- **extract_sender_info()** : Utilise email.utils.parseaddr pour separer nom et adresse depuis le champ From brut.
- **download_attachments()** : Telecharge les pieces jointes dans emails/attachments/ et retourne leurs chemins locaux.

Point notable

Le module charge egalement l'historique complet du thread Gmail via `get_thread_history()`, permettant a l'agent de tenir compte du contexte conversationnel.
La limite est de 200 messages par thread.

3.2 Etape 2 - ANALYSE (analyse.py)

Le module d'analyse utilise un LLM local (llama3 via Ollama, temperature=0) pour classifier l'email reçu et produire un objet DecisionModel.

Regles de classification

Intent	Criteres de detection	Action associee
calendar	L'email propose une date, demande une disponibilite ou veut planifier un RDV	Repond toujours reply
email	Question, demande d'information, echange standard sans notion de planification	Repond toujours reply
document	Email contenant des pieces jointes ou demandant l'envoi de fichiers	Action : review
general	Message informatif, notification sans action requise	Repond reply

unknown	Contenu ambigu ou insuffisant pour classifier	Action : review
---------	---	-----------------

Regles de priorite

- **urgent** : Urgence explicite, delai immediat mentionne
- **high** : Important avec contrainte de temps
- **normal** : Email standard sans indication de delai
- **low** : Message informatif sans urgence

Robustesse

La reponse du LLM est nettooyee (suppression des balises ``json`), puis parsee via `extract_json()`.
En cas d'echec total, un fallback safe est utilise :
{ intent: general, priority: normal, action: reply }

3.3 Etape 3 - AGENTS SPECIALISES

Le routage vers l'agent appropriee est effectuee dans `actions.py` par la fonction `route_to_agent()`, qui lit le champ `decision.intent` de l'`AgentState`.

Agent Email (`agent_email.py`)

Cet agent prend en charge les intents email, general et unknown. Il genere une reponse professionnelle contextualisee en s'appuyant sur deux niveaux de memoire :

- **Memoire court terme** : Historique du thread Gmail actuel - les messages les plus recents (jusqu'a 5 affiches en detail). Permet d'assurer la continuite d'une conversation en cours.
- **Memoire long terme** : Echanges passes avec le meme expéditeur, stockes dans ChromaDB (`chroma_email_db`). Recherche par similarite semantique (distance L2 ≤ 1.2) filteree par adresse email. Max 3 echanges injectes dans le prompt.

Prompt de generation

Le prompt guide strictement le LLM : il lui interdit d'inventer des informations, impose une structure obligatoire (Bonjour / corps / Cordialement), et limite la reponse a 3-4 phrases courtes.
Modele : llama3 (temperature=0.5, top p=0.80)

Agent Calendrier (`calendar_agent.py`)

Cet agent gere les demandes liees aux rendez-vous. Il distingue deux situations via `detect_email_type()` :

- **Nouvelle demande (`demande_rdv`)** : L'agent interroge Google Calendar, propose 2 creneaux libres en semaine (9h-16h), sauvegarde la proposition dans ChromaDB avec le statut pending, et envoie un email de proposition avec une reference unique.

- **Confirmation (confirmation_rdv)** : L'agent recherche la proposition en attente pour cet expéditeur, determine le creneau choisi, cree l'evenement dans Google Calendar avec invitation automatique, met a jour le statut ChromaDB de pending a confirmed.

Strategie de recherche RAG a 2 passes

1. Recherche semantique sur le contenu de l'email de confirmation (k=10 resultats)
 2. Filtrage strict : sender_email correspondant ET status=pending
- Si la recherche semantique echoue, un scan complet de la collection est effectue en fallback.

Agent RAG (agent_rag.py)

Cet agent permet de repondre a des questions basees sur des documents indexes. Il utilise ChromaDB avec des embeddings locaux via all-MiniLM-L6-v2 (SentenceTransformers).

- **get_relevant_docs()** : Recherche les 4 documents les plus proches par similarite cosinus. Seuls ceux avec un score ≤ 0.4 sont retenus.
- **rag_agent()** : Si des documents pertinents existent : repond en s'appuyant sur le contexte. Sinon : genere une reponse generale. Limite stricte de 150 tokens.

3.4 Etape 4 - ACTION (actions.py)

Ce module finalise le traitement de chaque email. Il orchestre 4 actions sequentielles :

- **1. Routage et generation** : Appel a route_to_agent() qui declenche l'agent approprié et retourne le texte de la reponse.
- **2. Envoi Gmail** : Utilise l'API Gmail pour envoyer la reponse. Le message est construit avec EmailMessage, encode en base64 URL-safe. Le thread_id est preserve. Le message original est marque comme lu.
- **3. Trace locale** : Sauvegarde un fichier texte dans emails/outbox/ au format sent_YYYYMMDD_HHMMSS.txt contenant expéditeur, sujet, intention, priorite, horodatage, reponse et email original.
- **4. Memoire vectorielle** : Stocke l'echange complet dans ChromaDB (chroma_email_db). Metadonnees indexees : sender_name, sender_email, subject, intent, priority, has_reply, timestamp.

4. Composants Transversaux

4.1 Authentification Google (gmail_auth.py)

Le module gere le flux OAuth2 pour Gmail et Google Calendar via google-auth-oauthlib. Scopes : gmail.readonly, gmail.send, gmail.modify et calendar.

- **token.json** : Stocke les credentials OAuth (access token + refresh token). Rechargement automatique a chaque demarrage.
- **credentials.json** : Fichier de configuration OAuth de l'application Google Cloud.
- **Renouvellement automatique** : Si le token est expire mais qu'un refresh_token est disponible, les credentials sont renouveles sans interaction utilisateur.

4.2 Indexation documentaire (rag_space/index_documents.py)

- **Formats supportes** : PDF, TXT, CSV, XLSX, DOCX, JSON
- **Suivi des modifications** : Un fichier JSON (indexed_files.json) maintient un hash MD5 de chaque document indexe. Un document n'est re-indexe que s'il a ete modifie.
- **Surveillance en temps reel** : Watchdog surveille le dossier documents/ et declenche l'indexation automatique des qu'un fichier est cree ou modifie.

4.3 Analyse des pieces jointes (attachments_analysis.py)

- **cv/** : Documents contenant : curriculum, competence, experience
- **facture/** : Documents contenant : facture, montant, tva
- **contrat/** : Documents contenant : contrat
- **document/** : Tout autre document non classifiable

4.4 Metriques de qualite (metrics.py)

- **Score orthographe** : Utilise LanguageTool (francais) pour detecter les fautes. Score = 1 - (erreurs / mots).
- **Score structure** : Verifie la presence de 'Bonjour' (+1/3), 'Cordialement' (+1/3) et une longueur minimale de 30 mots (+1/3).
- **Score global** : Moyenne des deux scores precedents, entre 0 et 1.

5. Interfaces Utilisateur

5.1 Dashboard Streamlit (ui_app.py)

- **Mode Automatique** : L'agent traite tous les emails non lus en un clic. Une case de confirmation est requise avant tout lancement.
- **Mode Manuel** : L'utilisateur sélectionne individuellement les emails à traiter. Traitement groupe possible.
- **Mode Visualisation** : Lecture seule, aucune action possible.

Les 4 onglets disponibles sont : Emails Non Lus, Emails Envoyés (50 derniers), Calendrier et interface l'Agent.

5.2 API REST FastAPI (app.py)

Endpoint	Description	Note
POST /api/agent/execute	Lance le pipeline complet	Execution manuelle
POST /api/rag/chat	Pose une question au RAG documentaire	Retourne la réponse LLM
POST /api/rag/upload	Upload et indexation d'un document	Indexation immédiate
GET /api/rag/documents	Liste des documents indexés	Fichiers supportés
GET /api/emails/unread	Emails non lus Gmail (50 max)	Metadonnées uniquement
GET /api/emails/sent	Emails envoyés Gmail (50 max)	Metadonnées uniquement
POST /api/emails/reply	Envoie un email manuellement	Corps libre
GET /api/calendar/events	Événements Google Calendar	Filtre par mois/année
GET /api/calendar/stats	Statistiques calendrier	Vue semaine/mois/année
GET /api/triggers/config	Configuration du trigger auto	Intervalle en minutes
POST /api/triggers/config	Active/désactive le trigger	Polling automatique

5.3 Visualiseur WebSocket (visualizer_server.py)

Un serveur FastAPI secondaire expose un endpoint WebSocket (/ws) permettant de suivre en temps réel la progression du pipeline LangGraph. Chaque noeud (PERCEPTION, ANALYSE, ACTION) est émis avec un délai de 0.7s pour une visualisation fluide.

6. Memoire et Persistance

6.1 Architecture de la memoire

Stockage	Contenu	Usage
chroma_email_db/	Echanges email (recu + reponse)	Recherche semantique par expéditeur
chroma_calendar_db/	Propositions de RDV	Suivi du statut (pending/confirmed)
rag_space/chroma_db/	Index documentaire RAG	Reponses aux questions metier
rag_space/indexed_files.json	Suivi des documents indexes	Hashes MD5

6.2 Cycle de vie de la memoire email

- **Ecriture** : A la fin de chaque traitement, `store_in_memory()` indexe l'echange dans `chroma_email_db` avec les metadonnees completes.
- **Lecture** : Au debut de la generation, `get_past_exchanges()` interroge ChromaDB. Filtre : similarite semantique ($L2 \leq 1.2$) + `sender_email` exact.
- **Utilisation** : Les echanges pertinents sont injectes dans le prompt section 'ECHANGES PASSES'. Max 3 echanges.

6.3 Cycle de vie de la memoire calendrier

- **Creation (status: pending)** : `save_proposal_to_rag()` stocke la proposition avec les deux creneaux proposes et un identifiant unique (UUID tronque a 8 caracteres).
- **Mise a jour (status: confirmed)** : `update_proposal_status()` supprime l'entree existante et la reinsere avec le nouveau statut.
- **Recherche** : `search_pending_proposal()` effectue une recherche semantique ($k=10$) puis filtre sur `sender_email` + `status=pending`.

7. Points Techniques Notables

7.1 Conception agentique avec LangGraph

Le pipeline est modelise comme un graphe oriente acyclique (DAG) avec LangGraph :

- START -> PERCEPTION -> ANALYSE -> ACTION -> END

Chaque noeud est une fonction Python qui recoit et retourne un AgentState. Le graphe est compile une seule fois au demarrage.

7.2 Gestion des imports circulaires

agent_email.py doit acceder a la collection ChromaDB creee dans actions.py, mais actions.py importe agent_email.py. Ce probleme est resolu par un import differe dans _get_memory_collection() : l'import d'actions n'est effectuee qu'au moment ou la collection est reellement necessaire.

7.3 Robustesse et gestion d'erreurs

- **Parsing JSON LLM** : Double protection avec nettoyage du markdown (``json) et extraction regex. Fallback garanti meme si le LLM genere du texte libre.
- **ChromaDB n_results** : Protection contre l'erreur 'n_results > nombre de documents disponibles' via un pre-comptage systematique.
- **Token OAuth expire** : Renouvellement automatique via refresh_token sans intervention utilisateur.

7.4 Limitations connues

- **Traitement sequentiel** : Le systeme ne traite qu'un email a la fois (unread_ids[:1]).
Limitation intentionnelle pour les tests.
mais il suffit mette en commentaire(91) (dans main_agent.py) cette ligne si on veut que tous les e-mails non lus soit traité

```
87  
88     # Récupération de tous les IDs non lus  
89     unread_ids = fetch_all_unread_ids()  
90  
91     unread_ids = unread_ids[:1]  
92
```

- **LLM local** : La qualite des reponses depend du modele llama3 disponible localement. Performance limitee par les ressources materielles.

8. Guide d'Utilisation

8.1 Prerequis

- **Ollama installé** : `ollama pull llama3`
- **credentials.json** : Application Google Cloud avec Gmail API et Calendar API activées
- **Authentification initiale** : `python gmail_auth.py` (une seule fois) pour générer `token.json`
- **Dependances Python**: `pip install -r requirements.txt`

8.2 Modes de lancement

Commande	Mode	Description
<code>python main_agent.py</code>	Agent CLI	Traite le 1er email non lu depuis le terminal
<code>python main_rag.py chat</code>	Chat RAG CLI	Session interactive de questions documentaires
<code>python main_rag.py index</code>	Indexation	Indexe tous les documents du dossier
<code>python main_rag.py watch</code>	Surveillance auto	Indexation automatique en temps réel
<code>uvicorn app:app --reload</code>	API FastAPI	Démarré l'interface REST + frontend web
<code>streamlit run ui_app.py</code>	Dashboard	Lance le tableau de bord graphique

8.3 Dossiers de données

- **emails/inbox/** : Emails de test au format texte (.txt)
- **emails/outbox/** : Traces locales des emails envoyés (`sent_YYYYMMDD_HHMMSS.txt`)
- **emails/attachments/** : Pièces jointes triées par catégorie (cv/, facture/, contrat/, document/)
- **rag_space/documents/** : Documents à indexer pour le RAG

9. Tests et Validation

9.1 Couverture de tests

Fichier de test	Module teste	Perimetre
test_generation.py	Agent Email	Generation de reponses LLM, qualite et coherence
test_reasoning.py	Module Analyse	Classification des intentions, regles de routage
test_calendar_agent.py	Agent Calendrier	Proposition et confirmation de RDV
test_gmail.py	Integration Gmail	Authentification OAuth, lecture et envoi

Lancement des tests

```
pytest tests/ -v
pytest tests/test_generation.py -v (tests specifiques)
pytest --tb=short (sortie condensee)
```

10. Conclusion et Perspectives

10.1 Bilan fonctionnel

Le systeme Multi-Agents realise l'ensemble de ses objectifs fonctionnels initiaux : lecture automatisee des emails, classification par intention, generation de reponses contextualisees, gestion autonome des rendez-vous et memoire long terme. L'architecture modulaire facilite la maintenance et l'extension du systeme.

10.2 Perspectives d'amelioration

- **Agent Document** : Implementer l'agent de traitement documentaire actuellement redirige vers l'Agent Email.
- **Traitement parallele** : Remplacer le traitement sequentiel par un pool de workers pour traiter plusieurs emails simultanement.
- **Modeles LLM avances** : Integrer des modeles plus puissants (llama3.1, mistral) pour ameliorer la qualite.
- **Interface de monitoring** : Ajouter des metriques de performance, tableaux de bord analytiques et alertes.
- **Apprentissage continu** : Implementer un mecanisme de feedback pour affiner les regles de classification.
- **Multi-boites** : Etendre le systeme pour gerer plusieurs comptes Gmail simultanement.

Document produit dans le cadre du projet academique Multi-Agents IA
Bilimbilim Mombo - Avril 2026